PROGRAMMING IN C (KTU - 2024 Scheme)

Common to Group A & B (Semester – 2)

C is often referred to as the 'mother of all programming languages.'

Module 1 : C Fundamentals

Module 2 : Arrays & Strings

Module 3 : Functions , Structures and Strings

Module 4 : Pointers and Files

A Brief History of C

C programming has a rich history that traces its roots back to earlier programming languages:

1. **ALGOL 60**: Introduced structured programming and influenced later languages.
2. **BCPL (Basic Combined Programming Language)**: Designed for writing system software.
3. **B**: A simplified version of BCPL developed by Ken Thompson.
4. **C**: Developed by Dennis Ritchie in 1972-73 at Bell Labs. It was created to rewrite the UNIX operating system and has since become one of the most widely used languages in the world.

The Need for Programming and the Role of Compilers

Why Do We Need Programming?

Programming is essential to interact with and harness the power of computers. However, computers operate in **binary code** (1s and 0s), which is impossible for humans to write or debug efficiently. Programming languages, such as C, Python, or Java, allow us to write instructions in a way that is easier for humans to understand.

The Role of Compilers in Bridging the Gap

A **compiler** is a special program that acts as a translator between humans and machines. It converts high-level programming code (written in human-readable languages like C) into machine code (binary) that a computer can execute. Here's why compilers are indispensable:

---

1. Translating Code

Compilers take your source code and translate it into a machine-readable format. For example:

Tech Talk

- High-level code: `printf("Hello, World!");`
- Compiled machine code: `01101000 01100101 ...` (binary instructions).

C Fundamentals

THE C CHARACTER SET

C uses the uppercase letters A to Z, the lowercase letters a to z, the digits 0 to 9, and certain special characters as building blocks to form basic program elements (e.g., constants, variables, operators, expressions, etc.).

Alphabets	A, B, ….., Y, Z a, b, ……, y, z
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special symbols	~ ‘ ! @ # % ^ & * ( ) _ - + = | \ { } [ ] : ; " ' < > , . ? /

C uses certain combinations of these characters, such as \b, \n and \t,to represent special conditions such as backspace, newline and horizontal tab, respectively. These character combinations are known as **escape sequences.**

TechTalkz

Constants (Literals) in C

In C programming, **constants** are fixed values that do not change during the execution of a program. Constants make your code more readable and easier to maintain, as they represent meaningful, unchanging data.

Types of Constants in C

1. **Integer Constants**
   - These are whole numbers without any fractional part.
   - Examples:
     - `10`
     - `-45`
     - `0`

   Rules for Integer Constants:

   - Must not have a decimal point.
   - Can be written in decimal (base 10), octal (base 8, starts with `0`), or hexadecimal (base 16, starts with `0x` or `0X`).

---

2. **Floating-point Constants**
   - These represent real numbers with fractional parts.
   - Examples:
     - `3.14`
     - `-0.001`
     - `2.5E3` (scientific notation for `2.5 × 10^3`).

   Rules for Floating-point Constants:

   - Must have at least one digit and a decimal point.
   - Can optionally include an exponent part (`E` or `e`).

---

3. **Character Constants**
   - A single character enclosed in single quotes (`' '`).
   - Examples:
     - `'A'`
     - `'9'`
     - `'#'`

   Rules for Character Constants:

   - Can represent printable characters, escape sequences (like `\n`, `\t`), or even ASCII values.

4. **String Constants**
   - A sequence of characters enclosed in double quotes (`" "`).
   - Examples:
     - `"Hello, World!"`
     - `"12345"`

TechTalkz

Rules for String Constants:

- Always ends with a null character (`\0`) automatically.
- Cannot span multiple lines without special techniques.

Identifiers

Identifiers are names given to various elements of a program, such as **variables**, **functions**, and **arrays**. They help in uniquely identifying these elements in the code.

Rules for Identifiers:

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)
- Names are case-sensitive (`myVar` and `myvar` are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (such as `int`) cannot be used as names

Examples of Valid Identifiers:

```
x           // Single letter
Y12         // Letters followed by digits
sum_1       // Letters, digits, and underscore
temperature // Readable name
names       // Simple word
area        // Descriptive
```

Examples of Invalid Identifiers:

Identifier	Reason
`8sum`	Starts with a digit.
`x'`	Contains an illegal character (`'`).
`order-no`	Contains a hyphen (-).
`error flag`	Contains a blank space.

TechTalkz

Best Practices for Identifiers:

1. Choose meaningful names to represent their purpose.
   Example:
   Instead of `x`, use `radius` if it represents the radius of a circle.
2. Avoid excessively long names.
   Example:
   Use `future_value` instead of `future_value_of_an_investment`.
3. Follow naming conventions, such as using lowercase letters for variables and underscores (_) to separate words.

Keywords

Keywords are reserved words in C with standard, predefined meanings. These cannot be used as identifiers.

List of Standard Keywords in C:

```
auto     extern   sizeof    break     float    static
case     for      struct    char      goto     switch
const    if       typedef   continue  int      union
default  long     unsigned  do        register void
double   return   volatile  else      short    while
enum     signed
```

TechTalkz